Pràctica 4: Comunicacions a través de la pila TCP/IP

Xarxes 25/26

Jan Accensi i Cristian Prado

Novembre 2025

Universitat de Barcelona

Índex

1 Introducció **2**

2 Realització pràctica: IoT i ThingSpeak **2**
2.1 Transmissió de dades genèrica 2
Q1 2
Q2 2
2.2 Ús de la llibreria ThingSpeak 3
Q3 4
2.2.1 Programa en Python per analitzar les dades 4
Q4 4
2.3 Arquitectura Client-Servidor i Punt d'Accés 5
Q5 5
Q6 5

3 Simulació amb Packet Tracer **6**
3.1 Configuració inicial i DHCP 6
Q7 6
Q8 7
3.2 Subnetting 8
Q9 8
Q10 8
3.3 Encaminament i anàlisi de tràfic 9
Q11 9
Q12 9
Q13 10
3.4 Connexió al núvol i protocols 11
Q14 11
Q15 11

4 Conclusions **12**

Annex A: Codi d'Anàlisi **13**

Annex B: Codi de generació d'un JSON **13**

Annex C: Codi JSON 10kB **14**

1 Introducció

Aquesta pràctica té com a objectiu principal aprofundir en el coneixement i l'aplicació de la pila de protocols TCP/IP, tant des d'una vessant pràctica d'implementació en dispositius reals com des d'una perspectiva de disseny i simulació de xarxes complexes. El desenvolupament s'estructura en dos grans blocs diferenciats però complementaris.

En la primera part, centrada en l'Internet de les Coses (IoT), treballem amb un microcontrolador ESP8266. L'objectiu és programar una arquitectura client-servidor capaç de monitoritzar la qualitat del senyal Wi-Fi (RSSI) i transmetre aquestes dades al núvol mitjançant la plataforma ThingSpeak. Aquest bloc ens permet analitzar la construcció de peticions HTTP çrues", la gestió de memòria en sistemes encastats mitjançant la fragmentació de dades JSON i l'ús de llibreries d'alt nivell per abstreure la complexitat del protocol.

La segona part es realitza mitjançant el simulador Cisco Packet Tracer per dissenyar una infraestructura de xarxa robusta. Es parteix d'una xarxa local simple per anar integrant serveis avançats com l'assignació dinàmica d'adreces (DHCP), la segmentació de xarxes (Subnetting) i la interconnexió de routers mitjançant enllaços sèrie. Finalment, la pràctica culmina amb la simulació d'una connexió WAN realista utilitzant el protocol Frame Relay i la configuració de NAT dinàmic per permetre l'accés a un servidor remot, proporcionant així una visió completa del trànsit de dades des del sensor fins al servidor corporatiu.

2 Realització pràctica: IoT i ThingSpeak

2.1 Transmissió de dades genèrica

Q1. Expliqueu que fan el conjunt d'instruccions que tenim dins la sentència if.

Bàsicament, aquest bloc de codi és el nucli de la comunicació amb el núvol. La sentència `if (client.connect( server, 80))` actua com a mecanisme de control: intenta establir una connexió TCP amb el servidor de ThingSpeak pel port 80 (HTTP). Si la connexió s'estableix correctament (retorna *true*), el programa entra al bloc per executar la transmissió de dades; en cas contrari, salta aquest pas per evitar errors d'execució.

Un cop dins del condicional, es realitzen tres tasques principals:

1. **Recollida i preparació de la dada:** Primer, es llegeix la potència del senyal Wi-Fi actual mitjançant la funció `WiFi.RSSI()`. Aquest valor es concatena en una cadena de text (`String body`) amb el format específic que espera l'API de ThingSpeak (del tipus `field1=-XX`).
2. **Construcció de la petició HTTP:** Com que en aquest apartat no fem servir la llibreria específica, el codi ha de construir manualment la capçalera del protocol HTTP línia per línia. S'especifica el mètode d'enviament (`POST`), s'identifica el servidor (`Host`) i, molt important, s'inclou l'autenticació mitjançant la clau d'escriptura a la capçalera `X-THINGSPEAKAPIKEY`. També es calcula i s'envia el `Content-Length`, necessari perquè el servidor sàpiga exactament quants bytes ocupa el missatge que rebrà.
3. **Enviament final:** Després d'enviar una línia en blanc (obligatòria en l'estàndard HTTP per separar les capçaleres del cos del missatge), s'envia la variable `body` amb la dada real. És en aquest moment quan la informació viatja efectivament cap a la base de dades de ThingSpeak per ser registrada.

Q2. Un cop explicada la funcionalitat del codi, expliqueu detalladament el seu funcionament i com interacciona amb el servidor de thingSpeak.

El funcionament del codi es basa en una arquitectura client-servidor sobre la pila TCP/IP, on l'ESP8266 actua com a client HTTP i la plataforma ThingSpeak com a servidor web i base de dades. El procés detallat d'interacció és el següent:

1. **Establiment de la sessió TCP:** El procés s'inicia amb la funció `client.connect(server, 80)`. Això estableix una connexió de transport (TCP) amb el servidor remot (`api.thingspeak.com`) a través del port 80. Aquest pas és previ a l'enviament de cap dada i garanteix que hi ha un canal de comunicació obert i fiable.
2. **Intercanvi HTTP (Mètode POST):** Un cop establert el canal, el dispositiu inicia una transacció HTTP utilitzant el verb `POST`. A diferència d'una petició `GET` (que sol · licita recursos), el `POST` indica al servidor que el client vol enviar dades per ser processades o emmagatzemades.

   La interacció amb el servidor es defineix mitjançant les capçaleres:

- **Autenticació:** La capçalera `X-THINGSPEAKAPIKEY` és crítica, ja que actua com a credencial d'accés. El servidor valida aquesta clau contra la seva base de dades; si és correcta, accepta l'escriptura al canal especificat.
- **Format de dades:** S'especifica el `Content-Type: application/x-www-form-urlencoded`, indicant al servidor com ha de "desempaquetar"el cos del missatge (`body`) per extreure'n el valor de l'RSSI.

3. **Processament i Tancament:** El servidor rep la petició, extreu el valor del camp `field1`, l'emmagatzema al canal associat a l'API Key amb una marca de temps (timestamp) i retorna un codi d'estat (normalment `200 OK`). Finalment, el client executa `client.stop()` per tancar el socket TCP, alliberant els recursos de xarxa del microcontrolador i finalitzant la sessió abans d'esperar al següent interval de mostreig.

2.2 Ús de la llibreria ThingSpeak

En primer lloc, mostrem el resultat que es mostra a la pàgina ThingSpeak, on hem obtingut un gràfic que va enviant les dades del RSSI que detecta la mota i un display numèric que mostra l'últim RSSI enviat:

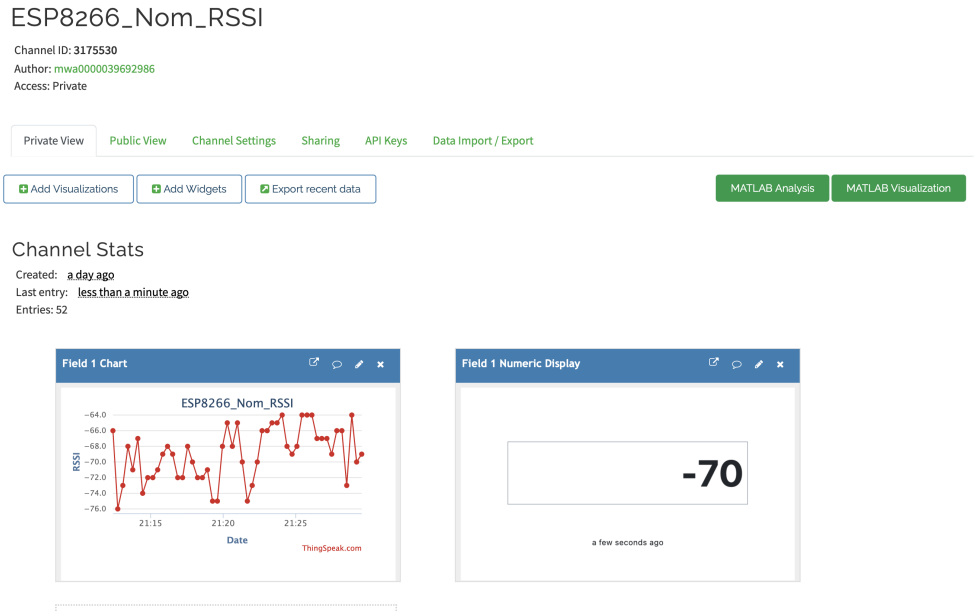

Figura 1: Chart i Numerical Display, estadístiques del canal

Per aconseguir la visualització de les dades mostrada a la imatge anterior, s'ha seguit el següent procediment tècnic basat en l'ús de la llibreria oficial de ThingSpeak:

1. **Configuració:** S'ha instal · lat la llibreria `ThingSpeak.h` a l'entorn Arduino IDE. Al codi, s'ha inicialitzat la comunicació dins el `setup()` mitjançant `ThingSpeak.begin(client)`. Dins del bucle principal, s'ha substituït la construcció manual de la petició HTTP per l'ús de les funcions `setField(1, rssi)` per assignar el valor i `writeFields(channelID, APIKey)` per realitzar l'enviament.
2. **Visualització al Dashboard:** Un cop la mota ha començat a transmetre, a la interfície web de ThingSpeak s'han configurat dos elements:
   - **Gràfic (Chart):** Es genera automàticament per defecte en detectar dades entrants al *Field 1*, mostrant l'evolució temporal de l'RSSI.
   - **Visualització Numèrica:** S'ha afegit manualment fent servir el botó **"Add Widget"** i seleccionant l'opció **"Numeric Display"**. S'ha configurat per mostrar l'últim valor rebut al *Field 1*, permetent una lectura ràpida i clara de la potència actual del senyal en dBm sense necessitat d'interpretar la gràfica.

Q3. Torna a resoldre les qüestions 1 i 2 però ara amb la llibreria de ThingSpeak. Si només has escollit fer-ho amb una o l'altra forma, respon a les preguntes només una vegada. Si implementes les dues, quines diferències i trobes?

En utilitzar la llibreria `ThingSpeak.h`, la lògica interna del codi i la interacció amb el servidor canvien significativament en nivell d'abstracció respecte a la implementació manual.

Revisió del funcionament (Q1 i Q2 amb llibreria):

L'ús de la llibreria transforma radicalment el flux de control del programa. A diferència del mètode manual, on el codi d'usuari és responsable d'obrir el sòcol TCP i construir la trama HTTP línia per línia, tota aquesta complexitat queda ara encapsulada dins la instrucció `ThingSpeak.writeFields()`. Aquesta funció actua com una caixa negra que gestiona automàticament l'establiment de la connexió amb el servidor, la generació de les capçaleres necessàries (incloent-hi l'autenticació amb l'API Key i el format de dades) i el tancament posterior de la sessió. A més, el mecanisme de validació evoluciona: en lloc de comprovar simplement si el servidor ha acceptat la connexió TCP, la funció retorna directament un codi d'estat HTTP (com el 200 per a èxit), permetent verificar que la transacció s'ha completat correctament en la capa d'aplicació.

Diferències principals trobades:

En comparar la implementació manual (mitjançant `WiFiClient` pur) amb l'ús de la llibreria, destaquen les següents diferències:

1. **Abstracció i Simplicitat:** La implementació manual requereix escriure explícitament el protocol HTTP ("POST /update...", "Host:...", Çontent-Length:..."). La llibreria abstreu tota aquesta "fontaneria", reduint unes 15 línies de codi de baix nivell a només 2 instruccions d'alt nivell (`setField` i `writeFields`), fent el codi molt més llegible i fàcil de mantenir.
2. **Escalabilitat:** Afegir nous paràmetres (per exemple, temperatura o humitat) en el mètode manual és complex, ja que implica re-calcular la longitud del missatge (`Content-Length`) i concatenar manualment la cadena de text del `body`. Amb la llibreria, només cal afegir una línia `ThingSpeak.setField(2, dada)` i ella s'encarrega de reestructurar el paquet i les capçaleres automàticament.
3. **Gestió d'Errors:** En el mètode manual, la gestió d'errors sol limitar-se a saber si s'ha pogut establir connexió o no. La llibreria, en canvi, retorna codis d'estat HTTP estandarditzats, facilitant la depuració en permetre distingir entre errors d'autenticació, errors de servidor o problemes de format.

2.2.1 Programa en Python per analitzar les dades

S'ha decidit generar també un gràfic de barres per poder veure la quantitat de vegades que s'han donat els valors RSSI. Per tant, s'ha realitzat un codi en Python que es pot consultar a l'Annex A. En primer lloc, es té una taula on recopilem tots els resultats que ens aporta el codi:

Paràmetre	**Valor**
Total mostres analitzades	100
Mitjana (μ)	-69.62 dBm
Desviació Estàndard (σ)	2.97 dB
Senyal Mínim	-80 dBm
Senyal Màxim	-64 dBm

Taula 1: Resultats de l'anàlisi estadística del senyal RSSI

Per altra banda, a continuació tenim el gràfic de barres que hem esmentat prèviament:

Q4. Explica i analitza el comportament complert del sistema desenvolupat: Els generadors de senyal RSSI i el recopilador de dades.

El sistema desenvolupat funciona com una cadena de telemetria completa, on podem distingir clarament els rols d'actuador i repositori. En primer lloc, el node ESP8266 actua com l'element actiu i determinista del sistema. La seva funció no és processar la informació, sinó capturar-la; cada 20 segons interroga el controlador Wi-Fi per obtenir el nivell de senyal i obre una connexió TCP per transmetre-ho immediatament. És important notar que el node no té memòria històrica; si la connexió falla, la dada es perd.

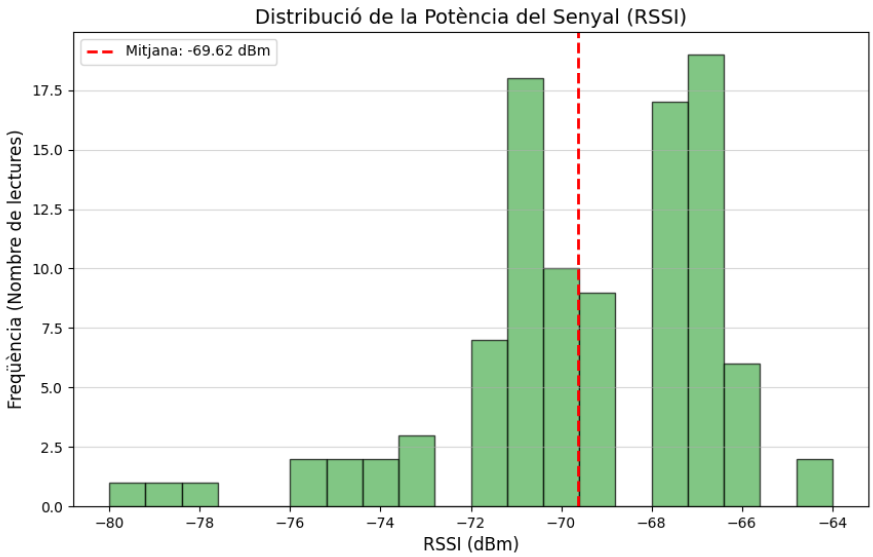

Figura 2: Gràfic de barres de RSSI

A l'altre extrem, ThingSpeak actua com un element passiu i reactiu. La seva missió principal és la validació i la persistència. En rebre la petició HTTP, verifica l'API Key i, si és correcta, assigna una marca de temps (*timestamp*) al paquet i el guarda a la base de dades. Això és el que ens permet transformar lectures instantànies i efímeres en un registre històric permanent.

Finalment, l'anàlisi realitzada amb l'script de Python (vegeu l'Annex A) ens permet interpretar la qualitat real de l'enllaç físic mitjançant dos paràmetres clau:

- **La Mitjana (μ):** Ens indica la potència "típica"de recepció. Un valor proper a -60 o -70 dBm es considera òptim, mentre que valors inferiors a -85 dBm indicarien que estem al límit de cobertura del punt d'accés.
- **La Desviació Estàndard (σ):** Aquest és l'indicador més crític per a la fiabilitat. Una desviació baixa indica un entorn net i estable, mentre que una desviació alta (pics de senyal) suggereix interferències externes, obstacles mòbils o congestió al canal Wi-Fi.

2.3 Arquitectura Client-Servidor i Punt d'Accés

Q5. Explica qui és, en el teu cas, el client i qui és el servidor. Perquè?

En la nostra implementació, els rols s'han definit clarament segons el model TCP/IP:

- **El Servidor (ESP8266):** La mota actua com a servidor per dos motius principals. Primer, a nivell d'infraestructura, genera la xarxa Wi-Fi (SSID) actuant com a Punt d'Accés (Mode AP). Segon, a nivell d'aplicació, obre un sòcol d'escolta al port 80 i es manté en espera passiva fins a rebre peticions, moment en el qual "serveix" les dades sol · licitades.
- **El Client (L'Ordinador/Consola):** El nostre ordinador actua com a client perquè és l'element actiu que inicia la comunicació. Primer es connecta a la infraestructura de xarxa creada per la mota i, posteriorment, realitza la petició HTTP explícita (en aquest cas, mitjançant la comanda `curl` des de la terminal) a l'adreça IP del servidor (192.168.4.1) per obtenir el recurs JSON.

El codi es pot trobar a l'Annex B.

Q6. Quina mida té el teu fitxer JSON? Genera un fitxer JSON de 10kBytes. Fragmenta'l en paquets de 1kByte i transmet-los de tal manera que les dades arribin correctament.

1. **Mida del fitxer inicial:** En la primera implementació del servidor (codi disponible a l'Annex B), el fitxer JSON generat dinàmicament té una mida exacta de **86 Bytes**. Aquesta mida s'ha verificat mitjançant un comptador de bytes intern al codi que suma la longitud de la capçalera i el contingut abans de tancar la connexió.

2. **Generació i fragmentació de 10 kBytes:** Per complir el segon requisit, s'ha modificat el servidor per generar un flux de dades de **10.250 Bytes** (aprox. 10 kB).

   Com que l'ESP8266 no disposa de suficient memòria RAM per emmagatzemar una cadena de text d'aquesta longitud, s'ha utilitzat una tècnica de transmissió fragmentada "al vol"(*on-the-fly*). L'algorisme (vegeu l'Annex C) funciona de la següent manera:

   - S'inicia la resposta HTTP i l'estructura JSON.
   - S'executa un bucle de 10 iteracions.
   - A cada iteració, es genera un bloc sintètic de 1024 bytes (1 kB) i s'envia immediatament al sòcol TCP.
   - S'introdueix un petit retard (50ms) entre iteracions per forçar que la pila TCP/IP enviï el paquet actual abans de generar el següent, garantint així la fragmentació correcta i evitant el desbordament de memòria.

Notem que per poder comunicar-nos, tant en el JSON curt com en el JSON llarg, es fa ús de la comanda `curl -v http://192.168.4.1`. L'ús de `curl -v` permet analitzar la transacció client-servidor en la seva forma més crua (*raw data*), descartant errors de capa d'aplicació del navegador. D'aquesta manera, es valida que la mota accepta connexions TCP i serveix les dades correctament segons l'estàndard HTTP/1.1. Per tant, es pot observar a la següent imatge que fent ús d'aquesta comanda s'han enviat correctament els paquets dels JSON llarg:

```
=== SERVIDOR 10KB LLEST ===
1. Connecta't a: Mota_Jan_Cristian_10KB
2. Obre IP: 192.168.4.1

[!] Client detectat via navegador.
 -> Paquet 1/10 enviat.
 -> Paquet 2/10 enviat.
 -> Paquet 3/10 enviat.
 -> Paquet 4/10 enviat.
 -> Paquet 5/10 enviat.
 -> Paquet 6/10 enviat.
 -> Paquet 7/10 enviat.
 -> Paquet 8/10 enviat.
 -> Paquet 9/10 enviat.
 -> Paquet 10/10 enviat.
========================================
TRANSMISSIO COMPLETADA.
MIDA EXACTA DEL FITXER JSON: 10376 Bytes
========================================
```

Figura 3: Paquets del JSON enviats correctament al client (ordinador)

3 Simulació amb Packet Tracer

3.1 Configuració inicial i DHCP

Q7. Quina comanda fem servir per reservar 10 adreces?

Per reservar un rang d'adreces IP i evitar que el servidor DHCP les assigni automàticament als clients, utilitzem la comanda global de configuració `ip dhcp excluded-address`.

En aquest cas concret, per reservar les 10 primeres adreces (de la 192.168.0.1 a la 192.168.0.10), la comanda executada al router és:

```
Router(config)# ip dhcp excluded-address 192.168.0.1 192.168.0.10
```

Aquesta instrucció s'ha d'introduir abans de definir el *pool* DHCP o immediatament després, però sempre fora de la configuració específica del pool. Això garanteix que els dispositius crítics (com servidors o impressores) que necessitin IP estàtica dins d'aquest rang no tindran conflictes amb els clients DHCP.

Q8. Fes una captura de pantalla on es mostri que hi ha connexió entre les dues interfícies del router explicant el procés de connexió.

En primer lloc, observem com queda la disposició de la xarxa:

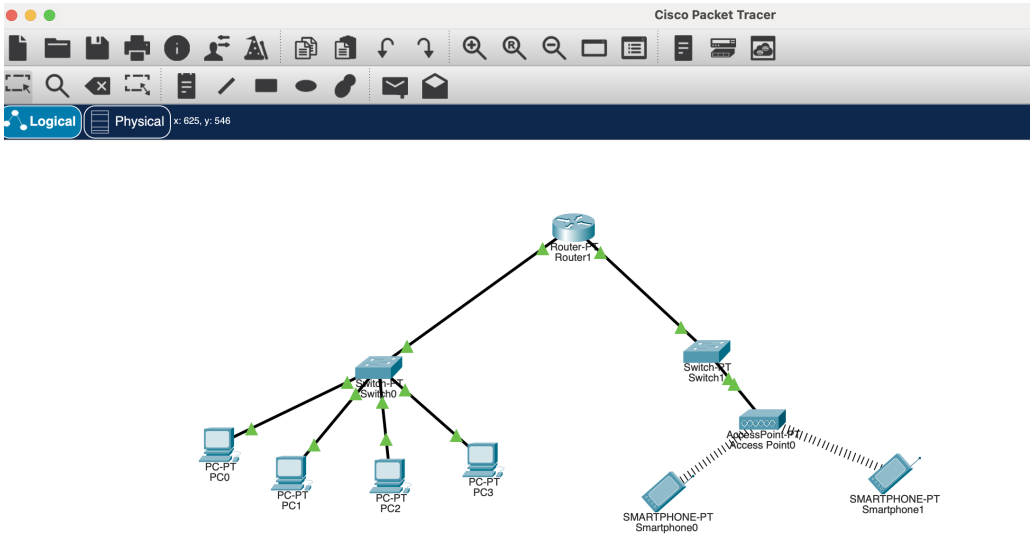

Figura 4: Disposició de la xarxa

La imatge següent mostra el resultat d'un test de connectivitat (`ping`) realitzat des d'un PC de la xarxa cablejada (`192.168.0.0/24`) cap a un dispositiu mòbil connectat a la xarxa sense fils (`192.168.1.0/24`). Notem que normalment el primer paquet acostuma a fallar perquè el router està aprenent la ruta, però posteriorment els altres paquets s'envien sense cap problema.

```
Cisco Packet Tracer PC Command Line 1.0
C:\>ping 192.168.1.1

Pinging 192.168.1.1 with 32 bytes of data:

Request timed out.
Reply from 192.168.1.1: bytes=32 time=33ms TTL=127
Reply from 192.168.1.1: bytes=32 time=42ms TTL=127
Reply from 192.168.1.1: bytes=32 time=20ms TTL=127

Ping statistics for 192.168.1.1:
    Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
Approximate round trip times in milli-seconds:
    Minimum = 20ms, Maximum = 42ms, Average = 31ms
```

Figura 5: Ping entre la xarxa cablejada i la xarxa sense fils.

Explicació del procés: La comunicació és possible gràcies a la funció d'encaminament (routing) del dispositiu central. El Router té dues interfícies actives connectades a xarxes diferents:

- Interfície `FastEthernet0/0`: Connectada a la xarxa `192.168.0.0/24` (Gateway `192.168.0.254`).
- Interfície `FastEthernet1/0`: Connectada a la xarxa `192.168.1.0/24` (Gateway `192.168.1.254`).

Quan el PC (origen) envia un paquet al Smartphone (destí), detecta que la IP de destí no pertany a la seva subxarxa local. Per tant, envia el paquet a la seva porta d'enllaç predeterminada (el Router). El Router rep el paquet, consulta la seva taula d'encaminament, veu que la xarxa de destí `192.168.1.0` està directament

connectada a la seva altra interfície i reenvia el paquet cap a l'Access Point, permetent així la interconnexió entre dominis de broadcast diferents.

3.2 Subnetting

Subnetting i Connectivitat

Q9. Per a què serveix el subnetting? Quantes IPs hi ha disponible per cada subxarxa?

El **subnetting** és la tècnica de dividir una xarxa IP física en múltiples xarxes lògiques més petites (subxarxes). Els seus objectius principals són:

- **Eficiència:** Redueix el malbaratament d'adreces IP assignant a cada xarxa només la quantitat necessària.
- **Seguretat i Organització:** Aïlla segments de xarxa (per exemple, separar departaments o plantes d'un edifici).
- **Rendiment:** Redueix la mida dels dominis de *broadcast*, millorant el rendiment general de la xarxa.

En el nostre cas, hem dividit una xarxa de classe C (/24) fent servir una màscara de subxarxa /25 (`255.255.255.128`). Això manlleva 1 bit a la part de host. Per tant, el que s'ha fet és tenir dues subxarxes que hem anomenat SUBNET_LEFT I SUBNET_RIGHT. La primera subxarxa comprén les IPs que van de 192.168.2.0 a la 192.168.2.127 inclosa. Notem que la IP 192.168.2.0 i 192.168.2.1 corresponen al *gateway* i a la xarxa. De forma anàloga tenim la segona subxarxa, que comprén les IPs que van de 192.168.2.128 a la 192.168.2.255 inclosa. Notem que la IP 192.168.2.128 i 192.168.2.129 corresponen al *gateway* i a la xarxa.

- **Càlcul d'IPs totals:** $2^7 = 128$ adreces per subxarxa.
- **IPs Utilitzables:** S'han de restar 2 adreces (la de xarxa i la de broadcast). Per tant, tenim **126 IPs disponibles** per a hosts a cada subxarxa.

Q10. Hi ha visibilitat entre les dues interfícies? En algun moment surt la IP 169.254.X.Y? Què vol dir aquesta adreça?

Visibilitat: Sí, hi ha visibilitat completa entre les dues interfícies i els PCs de totes dues subxarxes. Això és degut al fet que ambdues xarxes (`192.168.2.0/25` i `192.168.2.128/25`) estan **directament connectades** al mateix router. El router crea automàticament entrades a la seva taula d'encaminament (rutes *Directly Connected*) i permet el pas de paquets d'una interfície a l'altra sense necessitat de configuració addicional. A la imatge següent es pot observar que comprovem la visibilitat amb la comanda `ping`. En aquest cas, fem un ping des del PC amb IP 192.168.2.2 (de la SUBNET_LEFT) i executem `ping 192.168.2.130`, és a dir fem un `ping` amb un PC de l'altra subxarxa, la SUBNET_RIGHT.

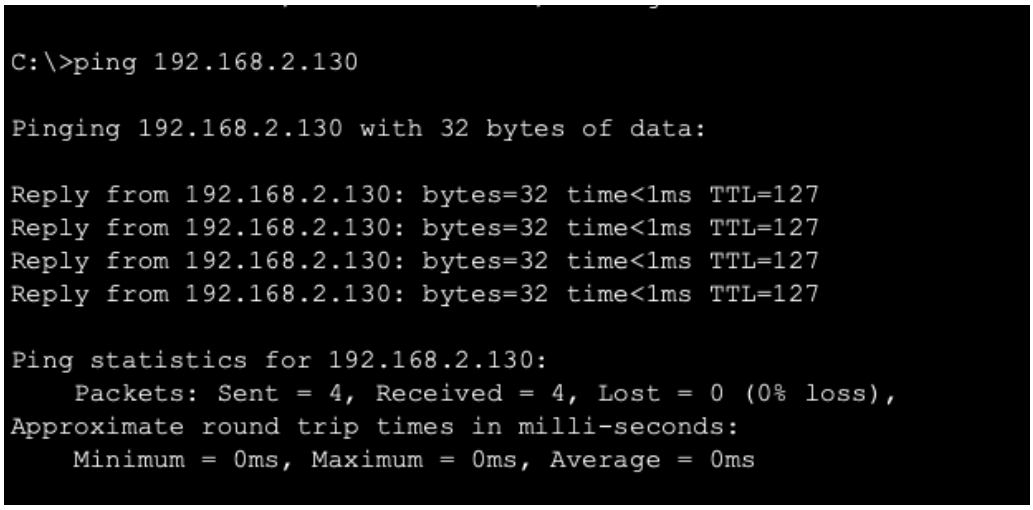

Figura 6: Comprovació visibilitat entre 2 PCs de diferents subxarxes

Sobre la IP 169.254.X.Y (APIPA): Durant el procés de configuració, si un PC encara no ha rebut resposta del servidor DHCP, és possible observar una IP del rang `169.254.X.Y`. Aquesta adreça correspon al protocol **APIPA** (Automatic Private IP Addressing). És un mecanisme de reserva que utilitzen els sistemes operatius

(Windows/Linux/Mac) quan tenen la interfície configurada en automàtic (DHCP) però **no aconsegueixen contactar amb cap servidor DHCP**. El sistema s'assigna una IP aleatòria d'aquest rang per permetre, almenys, la comunicació bàsica dins del mateix segment de xarxa local (Link-Local), encara que sense accés a Internet ni capacitat d'encaminament.

Tot i que en la simulació amb Packet Tracer de vegades es mostra simplement com a "DHCP request failed"(0.0.0.0), en un entorn real, quan un equip configurat per DHCP no rep resposta del servidor (per exemple, per una fallada de cablejat o configuració incorrecta del router), el sistema operatiu s'assigna automàticament una IP d'aquest rang (`169.254.0.0/16`). Això permet la comunicació bàsica d'enllaç local (Link-Local) entre dispositius del mateix segment que hagin fallat de la mateixa manera, però indica clarament una **manca de connectivitat amb el servidor DHCP i l'absència de sortida a Internet**.

3.3 Encaminament i anàlisi de tràfic

Q11. Explica detalladament quin és el mètode implementat i mostra el seu funcionament.

D'entre les dues opcions proposades a l'enunciat, s'ha escollit implementar l'**Encaminament Estàtic manual** mitjançant la comanda `ip route`.

Descripció del mètode: Aquest mètode consisteix a definir manualment a la taula d'encaminament de cada router quines són les xarxes remotes i quina és la "porta de sortida"per arribar-hi. A diferència del protocol RIP (que és dinàmic i intercanvia taules automàticament), l'encaminament estàtic no genera tràfic de control extra a la xarxa i proporciona un control total a l'administrador sobre el flux de dades.

Implementació i Funcionament: La interconnexió s'ha realitzat mitjançant les interfícies Sèrie (WAN) utilitzant el rang d'IPs públiques `161.116.95.0/24` proporcionat.

1. **Configuració de l'enllaç WAN:**
   - Router 0 (Interfície Sèrie): Configurat amb la IP `161.116.95.1`.
   - Router 1 (Interfície Sèrie): Configurat amb la IP `161.116.95.2`.
2. **Lògica d'Encaminament:**
   - **Al Router 0:** S'ha programat una ruta estàtica indicant que, per arribar a la subxarxa remota `192.168.2.0/24`, ha d'enviar els paquets a la IP del veí (`161.116.95.2`).
     `ip route 192.168.2.0 255.255.255.0 161.116.95.2`
   - **Al Router 1:** S'han programat rutes cap a les xarxes de l'esquerra (`192.168.0.0` i `1.0`), indicant que la porta de sortida és la IP del primer router (`161.116.95.1`).
     `ip route 192.168.0.0 255.255.255.0 161.116.95.1`

Respecte als elements que tenim al Packet Tracer, notem que fem ús de la connexió Serial DCE, connectant els dos routers a cada port Serial2/0. A partir d'això, com hem esmentat adalt, hem fet ús del CLI per poder configurar la interfície Serial2/0 de cada router. S'han configurat paràmetres com les adreces IP corresponents a cada interfície Serial2/0 i el clock rate. El clock rate només s'ha de configurar a un dels routers (s'ha escollit el router 0) i marca la velocitat a la que es transmeten les dades a través del cable.

El funcionament es verifica quan un PC de la xarxa local envia un paquet a l'altre extrem: el router consulta la seva taula estàtica, troba la coincidència manual i reenvia el paquet pel cable sèrie cap al següent salt definit.

Q12. Explica el funcionament d'aquesta comanda i el que passa quan l'executes

La comanda `ping` és l'eina fonamental de diagnòstic de xarxes. El seu funcionament es basa en el protocol **ICMP** (Internet Control Message Protocol) i serveix per verificar la connectivitat de capa 3 (IP) entre dos dispositius.

Seqüència d'execució i anàlisi: Quan executem la comanda cap a un ordinador de la xarxa remota, succeeix el següent procés:

1. **ICMP Echo Request (Petició):** L'ordinador origen genera un paquet ICMP de Tipus 8. Com que la IP de destí pertany a una subxarxa diferent, l'equip encapsula aquest paquet i l'envia a la seva **porta d'enllaç predeterminada** (la interfície del Router local).
2. **Encaminament (Routing):**

- El Router local rep la trama, desencapsula el paquet IP i consulta la seva taula d'encaminament. Gràcies a la ruta estàtica configurada (Q11), sap que ha d'enviar el paquet a través del port sèrie cap al Router veí.
- El Router remot rep el paquet, veu que la IP de destí correspon a una de les seves xarxes locals (Ethernet/Wi-Fi) i lliura el paquet al dispositiu final.

3. **ICMP Echo Reply (Resposta):** El dispositiu destí rep la sol · licitud i genera automàticament un paquet de resposta (ICMP Tipus 0), que fa el camí invers.

Resultat a l'Event List: A la llista d'esdeveniments del simulador, s'observa com el sobre de color negre (ICMP) viatja salt a salt entre els dispositius. Si la configuració és correcta, el terminal mostra el missatge `Reply from...` indicant el temps d'anada i tornada (RTT). Si la configuració d'encaminament fos errònia, rebríem un missatge de `Request timed out`.

Q13. Mostra el comportament i fes una captura de pantalla per veure que està passant.

Per analitzar detalladament el flux de dades, s'ha activat el mode de simulació del Packet Tracer aplicant un filtre exclusiu per al protocol ICMP. Aquesta acció permet aïllar el tràfic del `ping` d'altres missatges de control de la xarxa, com ara STP o CDP, facilitant-ne la interpretació.

Simulation Panel

Event List

Vis.	Time(sec)	Last Device	At Device	Type
	0.000	--	PC0	ICMP
	0.001	PC0	Switch0	ICMP
	0.002	Switch0	Router0	ICMP
	0.003	Router0	Router1	ICMP
	0.004	Router1	Switch2	ICMP
	0.005	Switch2	PC4	ICMP
	0.006	PC4	Switch2	ICMP
	0.007	Switch2	Router1	ICMP
	0.008	Router1	Router0	ICMP
	0.009	Router0	Switch0	ICMP
	0.010	Switch0	PC0	ICMP
	1.010	--	PC0	ICMP
	1.011	PC0	Switch0	ICMP
	1.012	Switch0	Router0	ICMP
	1.013	Router0	Router1	ICMP
	1.014	Router1	Switch2	ICMP
	1.015	Switch2	PC4	ICMP
	1.016	PC4	Switch2	ICMP
	1.017	Switch2	Router1	ICMP
	1.018	Router1	Router0	ICMP
	1.019	Router0	Switch0	ICMP
	1.020	Switch0	PC0	ICMP
	2.022	--	PC0	ICMP
	2.023	PC0	Switch0	ICMP
	2.024	Switch0	Router0	ICMP
	2.025	Router0	Router1	ICMP
	2.026	Router1	Switch2	ICMP
	2.027	Switch2	PC4	ICMP
	2.028	PC4	Switch2	ICMP
	2.029	Switch2	Router1	ICMP
	2.030	Router1	Router0	ICMP
	2.031	Router0	Switch0	ICMP
	2.032	Switch0	PC0	ICMP
	3.034	--	PC0	ICMP
	3.035	PC0	Switch0	ICMP

Figura 7: Seqüència d'intercanvi de paquets ICMP a l'Event List.

Com s'aprecia a la Figura 7, l'execució de la comanda desencadena un intercanvi seqüencial de missatges on cada línia representa un esdeveniment de transmissió. El procés s'inicia quan l'equip origen emet una petició *Echo Request* (ICMP Tipus 8) que, en ser rebuda i processada pel destinatari, genera una resposta immediata *Echo Reply* (ICMP Tipus 0). Aquesta informació es pot obtenir clicant als quadrats negres que veiem a l'Event List i és la informació PDU al dispositiu d'aquell moment. En particular podem saber el tipus el ICMP al time 0.000 i al 0.005, corresponent al primer paquet del `ping`. A més, es pot obbservar clarament la jerarquia de protocols. A la Capa 3 (Xarxa), les adreces IP d'origen i destí es mantenen invariables durant tot el trajecte, identificant els extrems de la comunicació. En canvi, a la Capa 2 (Enllaç de dades), l'adreça MAC de destí no correspon a l'equip final, sinó a la interfície del router local (Gateway). Això evidencia el mecanisme d'encaminament IP: la trama Ethernet lliura el paquet salt a salt fins al router, mentre que el datagrama IP dirigeix la informació fins al destí final.

3.4 Connexió al núvol i protocols

Q14. Identifiqueu els diferents protocols involucrats en la comunicació entre servidor i client.

Fent servir el mode de simulació (*sniffer*) del Packet Tracer per analitzar una petició HTTP des d'un PC client cap al servidor `192.168.3.10`, s'identifica l'actuació coordinada dels següents protocols a través de la pila TCP/IP:

- **Capa d'Aplicació (HTTP):** És el protocol visible. El navegador del client genera una petició `GET / HTTP/1.1` per obtenir la pàgina web, i el servidor respon amb el codi HTML. A diferència de l'Arduino, aquí el navegador gestiona automàticament capçaleres complexes (User-Agent, Accept, etc.).
- **Capa de Transport (TCP):** La comunicació és fiable i orientada a connexió. Abans de l'intercanvi de dades web, s'observa el procés de *Three-Way Handshake* (paquets SYN, SYN-ACK, ACK) per establir la sessió al port 80 del servidor.
- **Capa de Xarxa (IP i NAT):**
  - **Encaminament:** Els paquets viatgen basant-se en les IPs lògiques.
  - **NAT (Network Address Translation):** S'observa un canvi crític en les capçaleres IP quan el paquet travessa el Router 0. La IP privada d'origen (`192.168.0.x`) és substituïda per una IP pública del pool configurat (`161.116.95.x`), permetent que la petició sigui encaminable a través de la xarxa pública simulada.
- **Capa d'Enllaç (Ethernet i Frame Relay):** Aquest és el punt on la topologia és més complexa:
  - Dins de les xarxes locals (LAN del client i LAN del servidor), s'utilitzen trames **Ethernet II** amb adreçament MAC.
  - En travessar els routers cap al núvol (WAN), el protocol d'encapsulació canvia a **Frame Relay**. Aquí, les adreces MAC desapareixen i s'utilitzen els identificadors de circuit virtual **DLCI** (per exemple, DLCI 101 o 102) per commutar els paquets a través de la infraestructura del núvol fins al router de destí.

Q15. A partir d'aquesta informació, compareu els protocols i els resultats amb la tasca feta en la primera part de la pràctica.

En comparar la implementació IoT amb l'ESP8266 (Part 1) i la simulació de xarxa corporativa amb Packet Tracer (Part 2), s'observen diferències fonamentals en l'abstracció i la infraestructura:

1. **Control del Protocol (Manual vs. Automàtic):**
   - A la **Part 1 (Arduino)**, actuàvem com a programadors de baix nivell. Havíem de construir manualment la petició HTTP, definint explícitament el mètode `POST` o calculant el `Content-Length` byte a byte. El control sobre les dades enviades era total, però la complexitat de gestió era alta.
   - A la **Part 2 (Packet Tracer)**, actuem com a administradors de xarxa. El protocol HTTP el genera automàticament el navegador web "client". La nostra tasca no és definir *què* s'envia, sinó garantir *per on* passa (configurant rutes i NAT).
2. **Visibilitat de la Infraestructura (Caixa Negra vs. Caixa Blanca):**
   - En la part de l'ESP8266, la xarxa externa era una "caixa negra". Ens connectàvem a una Wi-Fi i les dades sortien a Internet sense que sapiguéssim com es feia la traducció d'adreces o l'encaminament.
   - En la simulació, hem obert aquesta caixa. Hem configurat manualment el **NAT Dinàmic** i l'encapsulació **Frame Relay**, fent visible tota la complexitat intermèdia (traducció d'IPs privades a públiques i commutació de circuits WAN) que normalment queda oculta a l'usuari final o al desenvolupador d'aplicacions.
3. **Tecnologia d'Accés:** Mentre que la primera part es basava exclusivament en l'estàndard inalàmbric **802.11 (Wi-Fi)** per a l'accés al medi, la segona part il · lustra un entorn híbrid real on conviuen Ethernet (cablejat), Wi-Fi (mòbils) i protocols WAN (sèrie/Frame Relay), demostrant com la pila TCP/IP unifica la comunicació independentment del mitjà físic subjacent.

4 Conclusions

La realització d'aquesta pràctica ha permès consolidar els conceptes teòrics sobre la comunicació de dades, evidenciant la complexitat subjacent en processos que habitualment són transparents per a l'usuari final.

Pel que fa a la implementació amb l'**ESP8266**, s'ha comprovat la versatilitat del dispositiu actuant sota diferents rols. Com a client, hem vist la importància de gestionar correctament les capçaleres HTTP i l'autenticació (API Keys) per interactuar amb serveis al núvol com ThingSpeak. Com a servidor (Punt d'Accés), s'ha demostrat com les limitacions físiques del maquinari (poca memòria RAM) obliguen a utilitzar estratègies de programació eficients, com la generació de respostes JSON fragmentades "al vol" per transmetre grans volums de dades sense saturar el sistema. Així mateix, l'ús comparatiu de llibreries (`ThingSpeak.h`) versus codi natiu ha posat de manifest l'equilibri necessari entre facilitat de desenvolupament i control absolut sobre el protocol.

En l'àmbit de la **simulació amb Packet Tracer**, hem pogut aixecar la "caixa negra" de la infraestructura d'Internet. La configuració manual de l'encaminament estàtic i, especialment, la implementació de la xarxa WAN amb **Frame Relay** i **NAT Dinàmic**, ha estat clau per entendre com es garanteix la connectivitat global. L'ús d'eines d'anàlisi de paquets (*sniffer*) ha estat fonamental per verificar com la informació s'encapsula i es transforma (de trama Ethernet a trama Frame Relay, i d'IP privada a IP pública) a mesura que travessa els diferents segments de la xarxa.

En conclusió, hem integrat amb èxit els dos extrems de la comunicació: des del sensor que genera la dada "crua" fins a la complexa infraestructura d'encaminament i traducció que permet que aquesta dada arribi al seu destí a través de la xarxa pública.

Annex A: Codi d'Anàlisi de Dades

A continuació es mostra l'script en Python utilitzat per processar el fitxer `dades_rssi.csv`, netejar les dades i calcular els paràmetres estadístics sol · licitats.

```python
import pandas as pd
import matplotlib.pyplot as plt

# ==============================================================================
# CONFIGURACIO
# ==============================================================================
# Nom del fitxer descarregat de ThingSpeak
NOM_FITXER = 'dades_rssi.csv'

def analitzar_dades():
    try:
        print(f"Llegint dades de {NOM_FITXER}...")

        # 1. Carreguem les dades
        df = pd.read_csv(NOM_FITXER)

        # 2. Neteja de dades (RSSI a la columna 'field1')
        df['field1'] = pd.to_numeric(df['field1'], errors='coerce')
        dades_netes = df.dropna(subset=['field1'])

        if len(dades_netes) == 0:
            print("Error: No s'han trobat dades valides.")
            return

        # 3. Calculs Estadistics
        mitjana_rssi = dades_netes['field1'].mean()
        desviacio_estandard = dades_netes['field1'].std()
        minim = dades_netes['field1'].min()
        maxim = dades_netes['field1'].max()

        # 4. Mostrar Resultats
        print("\n" + "="*40)
        print("       RESULTATS DE L'ANALISI RSSI")
        print("="*40)
        print(f" Total mostres analitzades: {len(dades_netes)}")
        print(f" -> Mitjana (Mean):         {mitjana_rssi:.2f} dBm")
        print(f" -> Desviacio Estandard:    {desviacio_estandard:.2f} dB")
        print(f" -> Minim senyal registrat: {minim} dBm")
        print(f" -> Maxim senyal registrat: {maxim} dBm")
        print("="*40 + "\n")

        # 5. Generar Grafic
        plt.figure(figsize=(10, 6))
        plt.hist(dades_netes['field1'], bins=20, color='#4CAF50', edgecolor='black')
        plt.title('Distribucio de la Potencia del Senyal (RSSI)')
        plt.xlabel('RSSI (dBm)')
        plt.ylabel('Frequencia')
        plt.axvline(mitjana_rssi, color='red', linestyle='dashed', label='Mitjana')
        plt.legend()
        plt.show()

    except Exception as e:
        print(f"Error: {e}")

if __name__ == "__main__":
    analitzar_dades()
```

Listing 1: Script Python per al càlcul de mitjana i desviació estàndard

Annex B: Codi de generació d'un JSON curt i càlcul de mida

```boilerplate
#include <ESP8266WiFi.h>

// Configuracio de la xarxa
const char *ssid = "Mota_Jan_Cristian_Curt";
const char *password = "12345678";

WiFiServer server(80);

```

```
void setup() {
  Serial.begin(9600);
  delay(10);

  // Configuracio AP
  WiFi.mode(WIFI_AP);
  WiFi.softAP(ssid, password);
  server.begin();

  Serial.println("\n=== SERVIDOR JSON CURT INICIAT ===");
  Serial.print("1. Connecta't a: "); Serial.println(ssid);
  Serial.print("2. Obre IP: "); Serial.println(WiFi.softAPIP());
}

void loop() {
  WiFiClient client = server.available();

  if (client) {
    Serial.println("\n[!] Client detectat.");

    // Esperem peticio i netegem buffer
    while (!client.available()) delay(1);
    client.flush();

    // 1. Capcaleres HTTP (El sobre, no compten per la mida del fitxer)
    client.println("HTTP/1.1 200 OK");
    client.println("Content-Type: application/json");
    client.println("Connection: close");
    client.println();

    // Variable per comptar la mida del JSON
    unsigned long midaBytes = 0;

    // 2. Creacio del contingut JSON
    // Creem un missatge senzill
    String jsonContent = "{\n";
    jsonContent += "  \"estat\": \"Connectat\",\n";
    jsonContent += "  \"missatge\": \"Aixo es un JSON curt\",\n";
    jsonContent += "  \"valor_random\": " + String(random(0, 100)) + "\n";
    jsonContent += "}";

    // 3. Enviament
    client.print(jsonContent);

    // 4. Calcul de la mida (Metode 1)
    midaBytes = jsonContent.length();

    delay(10);
    client.stop();

    // 5. Mostrem el resultat pel Monitor Serie
    Serial.println("--------------------------------------");
    Serial.println("Transmissio finalitzada.");
    Serial.print("MIDA EXACTA DEL FITXER JSON: ");
    Serial.print(midaBytes);
    Serial.println(" Bytes");
    Serial.println("--------------------------------------");
  }
}
```

Listing 2: Codi per a la generació d'un JSON curt i càlcul de mida

Annex C: Codi de Generació de JSON de 10kB i Fragmentació

Aquest codi implementa l'estratègia de generació "al vol"(*on-the-fly*) per transmetre un flux de dades JSON de 10 kB sense desbordar la memòria RAM del microcontrolador. També inclou el comptador de bytes per verificar la mida total transmesa.

```
#include <ESP8266WiFi.h>

// Configuracio de la xarxa
const char *ssid = "Mota_Jan_Cristian_10KB";
const char *password = "12345678";

```

```
WiFiServer server(80);

void setup() {
  Serial.begin(9600);
  delay(10);

  // Configuracio AP
  WiFi.mode(WIFI_AP);
  WiFi.softAP(ssid, password);

  server.begin();

  Serial.println("\n=== SERVIDOR 10KB LLEST ===");
  Serial.print("1. Connecta't a: "); Serial.println(ssid);
  Serial.print("2. Obre IP: "); Serial.println(WiFi.softAPIP());
}

void loop() {
  WiFiClient client = server.available();

  if (client) {
    Serial.println("\n[!] Client detectat via navegador.");

    // Esperem la peticio i netegem buffer
    while (!client.available()) delay(1);
    client.flush();

    // 1. Capcaleres HTTP
    client.println("HTTP/1.1 200 OK");
    client.println("Content-Type: application/json");
    client.println("Connection: close");
    client.println();

    // Variable per comptar la mida real
    unsigned long comptadorBytes = 0;

    // 2. Generacio del contingut
    // Part 1: Inici del JSON
    String inici = "{\n  \"titol\": \"Practica 4 Q6\",\n  \"dades_massives\": [\n";
    client.print(inici);
    comptadorBytes += inici.length();

    // Part 2: Generacio de 10 blocs de 1KB (Fragmentacio)
    // Creem un bloc de farciment de 1024 bytes
    String bloc1KB = "";
    for(int k=0; k<1024; k++) bloc1KB += "A";

    // Bucle per enviar-ho 10 vegades (10KB totals)
    for (int i = 0; i < 10; i++) {
      client.print("    \"");
      client.print(bloc1KB);    // El bloc d'1KB
      client.print("\"");

      // Sumem: 1KB + 2 bytes de cometes + 4 espais
      comptadorBytes += (bloc1KB.length() + 6);

      if (i < 9) {
        client.print(",\n"); // Coma si no es l'ultim
        comptadorBytes += 2;
      } else {
        client.print("\n");  // Salt de linia final
        comptadorBytes += 1;
      }

      // FRAGMENTACIO: Delay per assegurar enviament paquet a paquet
      delay(50);
      Serial.print(" -> Paquet "); Serial.print(i+1); Serial.println("/10 enviat.");
    }

    // Part 3: Tancament del JSON
    String finalJson = "  ]\n}";
    client.print(finalJson);
    comptadorBytes += finalJson.length();

    delay(100);
    client.stop();
```

```

    // Resultat pel Monitor Serie
    Serial.println("========================================");
    Serial.print("TRANSMISSIO COMPLETADA.\n");
    Serial.print("MIDA EXACTA DEL FITXER JSON: ");
    Serial.print(comptadorBytes);
    Serial.println(" Bytes");
    Serial.println("========================================");
  }
}
```

Listing 3: Generació i fragmentació d'un JSON de 10kB